

生成式软件开发-回顾

– 从 Prompt 到 Context到 Harness 的演进

大纲

在课程的上半学期，我们深入探讨了如何利用**大型语言模型（LLM）**进行软件开发与任务自动化。

随着工具链和应用场景的复杂化，我们的视角也经历了从单一的提示词构建，到复杂的**系统性上下文管理**，再到**平台级治理**的演进。

01 基于浏览器会话的提示词工程
Prompt Engineering

02 基于 Agentic IDE 的上下文工程
Context Engineering

03 Plan 模式开发的最佳实践
Spec-Driven Development

04 Agent Skills 的设计与实现
Reusable Knowledge Components

05 上下文工程归纳
Unified Methodology Framework

06 下一步：从 Prompt 到 Context 到 Harness
Evolution Path & Future

核心洞察： 讲义将对上半学期的核心知识点进行系统性回顾，并梳理技术演进的主线。

01

基于浏览器会话的提示词工程

Prompt Engineering

将"模糊的想法"翻译成"可执行的指令"——生成式软件开发的起步阶段

提示词工程

角色设定

Role Prompting

通过设定专业角色（如"资深前端工程师"），收敛上下文范围，确立代码输出的专业标准。

结构化提示词

Structured Prompting

利用 Markdown 或 XML 标签将需求结构化（如分为背景、目标、功能需求），使边界清晰，降低模型解析难度。

提供示例

N-Shot Prompting

通过单样本(One-shot)或少样本(Few-shot)演示，"Show, don't just tell"，帮助模型快速模仿特定的逻辑、格式或代码风格。

思维链

Chain-of-Thought, CoT

要求模型"一步步思考"，在输出最终代码前先进行步骤拆解和方案对比，显著提升复杂逻辑和算法设计的准确率。

⚠ Chat 模式的局限性

尽管门槛低、起步快，但缺乏对本地文件系统的感知（代码分散、需手动粘贴）、调试繁琐，且跨会话无法继承上下文。这自然引出了向 **Agentic IDE** 演进的需求。

02

基于 Agentic IDE 的上下文工程

Context Engineering

从单点脚本到复杂多文件项目——工作闭环的形成与上下文管理的核心方法论

上下文工程

随着开发任务从"单点脚本"向"复杂多文件项目(Project)"演进，Agentic IDE 引入了 Workspace（工作区）的概念。开发模式由简单的问答升级为具备自主执行能力的**工作闭环**：任务拆分 → 读取上下文 → 实施改动 → 执行验证 → 基于反馈迭代 → 交付沉淀。

规则上下文
Rule Context

通过 `.trae/rules` 等配置文件，固化项目的"基因"（如指定 Python+Streamlit 技术栈、约定中文注释和多文件架构）。这确保了 AI 在整个项目生命周期内输出的一致性。

规划层上下文
Planning Context

通过维护 `Spec.md`（需求规格与接口定义）和 `Todo.md`（任务进度管理），将复杂系统拆解为可执行的模块，明确验收标准，从而支持长周期的渐进式交付。

手动上下文
Manual Context

开发者在下达指令时，通过 `@ 文件`（如引入 `@settings.py`）精确提供高信噪比的局部依赖，消除歧义。

自动上下文
Auto Context

依托 Agentic IDE 的全局索引能力，AI 能够自主感知 Workspace 中的代码结构（全局感知），并在修改时自动进行跨文件调度（多文件联动）。

核心认知转变

在 Agentic IDE 模式下，高效的生成式开发本质上就是**高效的上下文管理**。通过系统化地建立规则、规划任务并动态传递相关知识，我们能够引导 AI 稳定地完成复杂的工程架构。

03

Plan 模式开发的最佳实践

Spec-Driven Development

规范驱动开发：先规划，后编码——解决上下文漂移与系统熵增问题

Plan 模式

随着项目复杂度的提升，即便是拥有全局上下文的 Agentic IDE，如果仅依赖琐碎的多轮对话（会话驱动），也会面临**上下文漂移（Lost in the Middle）**、倾向于"打补丁"而非重构、以及开发过程难以复现等瓶颈。为了解决这些问题，我们引入了 **Plan 模式（规范驱动开发）**。

先规划，后编码

将每一个步骤先转换为结构化的 Plan 文档，AI 基于"契约"生成代码，而非零散的对话记忆。这种前置规划确保了代码生成的系统性和一致性。

文档即契约 Spec-Driven

将每个步骤拆解为高层架构、数据模型、接口协议和业务逻辑等部分，通过文档明确边界和异常处理，建立不可篡改的安全基线。

Plan 模式的价值体现

抑制系统熵增

通过结构化规划避免代码库的混乱和腐化

提升确定性

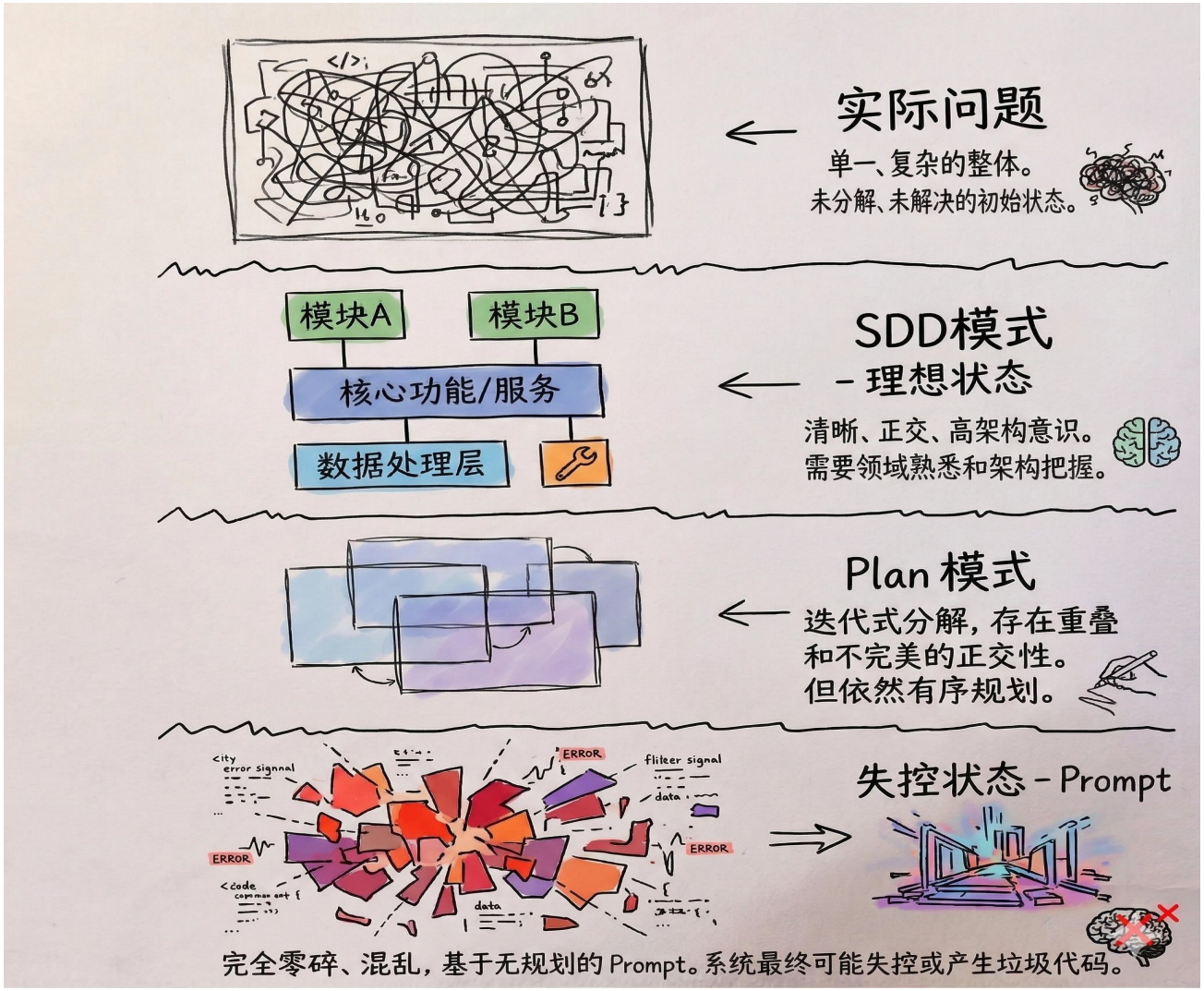
基于契约生成代码，确保输出的一致性和可预测性

增强可维护性

文档化的规划使代码变更可追溯、可审计

→ 通过 Plan 模式，我们有效地抑制了系统的熵增，让 AI 从**"灵活的聊天对象"**转变为**"基于严谨规范执行的工程助手"**。

Plan 模式示意图


实际问题
单一、复杂的整体。
未分解、未解决的初始状态。
模块A
模块B
核心功能/服务
数据处理层
SDD模式
- 理想状态
清晰、正交、高架构意识。
需要领域熟悉和架构把握。
Plan 模式
迭代式分解，存在重叠
和不完美的正交性。
但依然有序规划。
ERROR
ERROR
ERROR
data
失控状态 - Prompt
完全零碎、混乱，基于无规划的 Prompt。系统最终可能失控或产生垃圾代码。

04

Agent Skills 的设计与实现

Reusable Knowledge Components

Skills：可复用过程性知识的组件化封装——让上下文工程走向工具化

Skills 的机制与最佳实践

在解决了项目的规范化构建后，我们面临另一个问题：如何将特定的领域知识、个人经验和复杂工作流传授给 Agent，同时又不一直占用宝贵的上下文窗口？这就引入了 **Agent Skills**。

Skills 的核心定位

Skills 是大模型的"应用插件"，用简单、开放的文件夹格式（包含 `SKILL.md` 及相关脚本资源），把**可复用的过程性知识**封装起来。

它解决了大模型缺乏团队/个人特定上下文的问题，使任务执行变得**稳定、可审计和可复现**。

渐进式披露机制 Progressive Disclosure

① **发现** 通过元数据（`name` 和 `description`）让 Agent 发现可用技能

② **激活** 当任务匹配时，才加载 `SKILL.md` 正文，极大地节省了 Token

③ **执行** 按指令或脚本执行任务

Skills 的最佳实践

触发描述优化

`description` 是触发的关键，需包含能力、触发场景和关键词。通过"触发评测集"不断打磨描述。

适度细节与默认值

专注于 Agent 自己不知道的内容；给出明确的默认值而不是让其在众多选项中迷失。

高内聚单一职责

避免"职责过宽"的 Skill，复杂的任务应通过多个原子 Skill 或 Workflow Skill 组合完成。

结合自动化脚本

对于复杂的业务校验、数据处理或外部 API 交互，将逻辑封装在脚本中（如 Python、Bash），让 Agent 调用以提升确定性。

→ Skills 让上下文工程从**"文本提示"**进一步走向了**"工具化与组件化"**。

05

上下文工程归纳

Context Engineering Methodology

系统性管理AI信息环境的方法论体系——建立理论框架统一所有技术实践

上下文工程的理论框架与核心要素

上下文工程的定义

上下文工程 (Context Engineering) 是指：在大型语言模型(LLM)驱动的软件开发中，**系统性地收集、结构化、过滤、并动态注入与当前任务相关的背景信息**（知识、规则、状态、工具库），从而精确引导 AI 行为，抑制大模型幻觉与上下文漂移，确保输出质量与工程一致性的方法论体系。

四大核心要素

01 边界控制

知道**给什么**（高信噪比信息）与**不给什么**（防止注意力稀释）。精确控制输入上下文的范围和质量。

02 结构化表达

用人机皆可读的格式（如 Markdown、YAML、JSON）传递意图。结构化的信息更容易被模型理解和执行。

03 动态路由

基于当前任务进度，**按需加载**（渐进式披露）。避免一次性加载过多无关信息，优化 Token 使用效率。

04 持久化沉淀

将零散的知识转化为**可版本控制的工程资产**（文件、脚本）。确保知识的可追溯性和团队协作的一致性。

四类技术实践的内在联系（一）

上述提到的所有技术实践，本质上都是在**不同维度、不同颗粒度上解决上下文传递问题**的具体表现形式。

01 基于浏览器会话的 Prompt Engineering

任务级上下文

机制与原理

在最基础的 Chat 模式中，上下文是通过对话历史和当前输入来构建的。

作为上下文工程的表现

通过**角色设定**和**结构化提示词**，我们在单次会话的生命周期内，手动为大模型建立了一个临时且脆弱的微观上下文环境。

价值体现

适合快速原型验证，是上下文工程的启蒙。

02 基于 Agentic IDE 的 Context Engineering

工程级上下文

机制与原理

将大模型接入真实的文件系统（Workspace），引入了自动索引。

作为上下文工程的表现

通过 `.trae/rules` 建立**规则上下文**，通过 `@文件` 引入**手动依赖上下文**。AI 拥有了整个代码库的感知能力。

价值体现

上下文从"临时记忆"升级为"本地持久化存储"，使得多文件联动和复杂项目的修改成为可能。

四类技术实践的内在联系（二）

03 Plan 模式开发的最佳实践

规划与状态级上下文

机制与原理

通过编写 `Spec.md` 、`Architecture.md` 等架构文档，将人的思考过程剥离出来。

作为上下文工程的表现

这是针对复杂系统的**时间/状态上下文管理**。通过将需求变化反映到 Plan 文档中（变更驱动文档），避免了多轮会话导致的"上下文漂移"，建立了不可篡改的安全基线（双 阶段提交）。

价值体现

为 AI 提供了一致的目标和契约，使代码生成具备极高的确定性和可维护性。

04 Agent Skills 的设计与实现

领域与能力级上下文

机制与原理

将特定领域的工作流、避坑指南、自动化脚本打包成一个带有 `SKILL.md` 的独立文件夹。

作为上下文工程的表现

这是**过程性知识的上下文封装**。它利用**渐进式披露**（Progressive Disclosure）技术，在 Discovery 阶段仅通过 Metadata 暴露能力，只有在 Activation 阶段才将完整的指令注入到上下文窗口中。

价值体现

极大地节省了 Token 开销，让 Agent 拥有了"外挂大脑"和"特定团队基因"，实现了能力的组件化和跨环境复用。

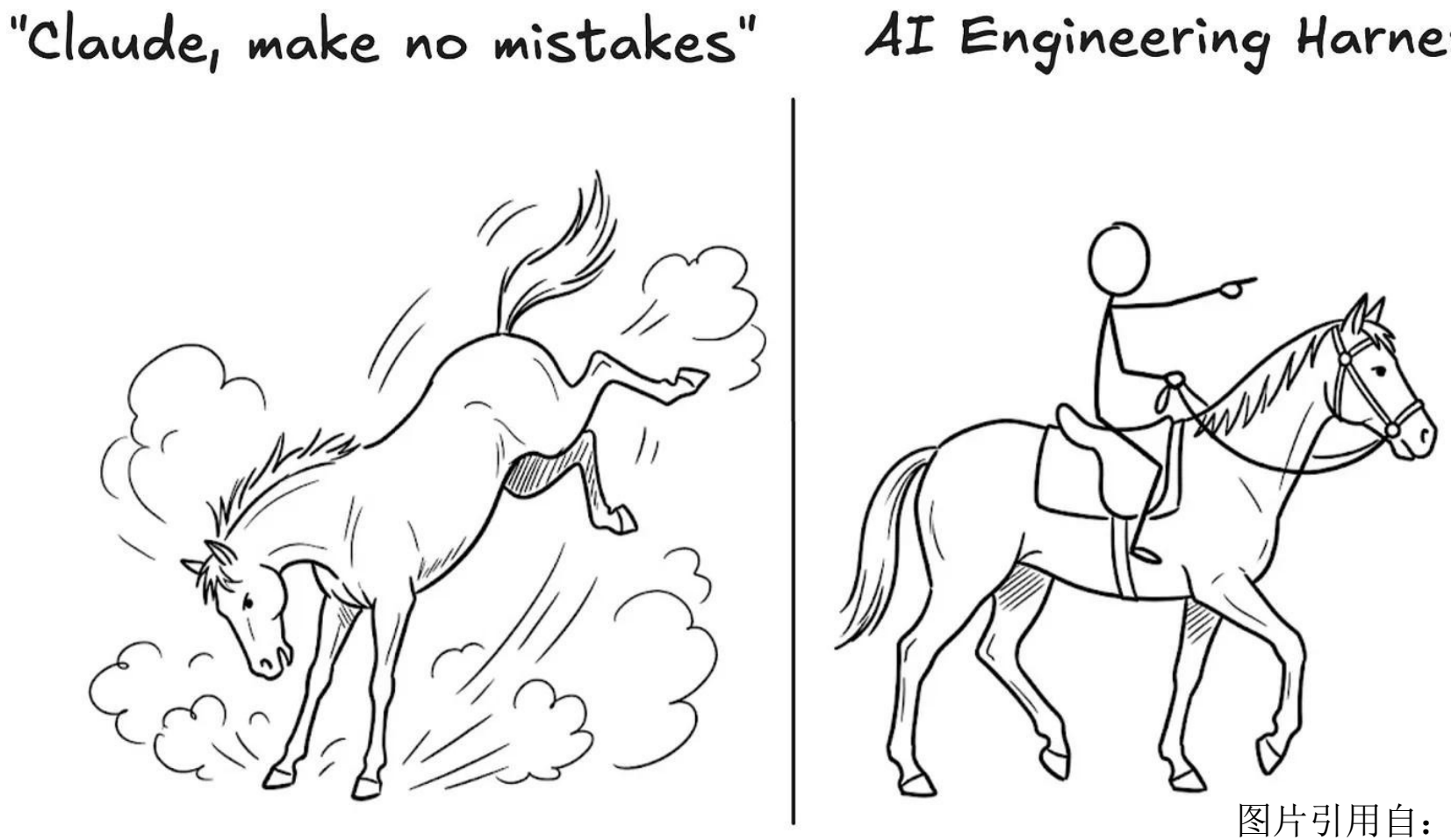
06

从 Prompt 到 Context 到 Harness

Evolution Path: From Conversation to System

范式转移：从对话到系统，构建可靠的生产级 AI Agent

Harness是什么



图片引用自：
strategizeyourcareer.com

演进路径

在大型语言模型辅助软件开发的过程中，我们正在经历一次从"对话"到"系统"的范式转移，而构建可靠的生产级 AI Agent 最终需要的是 **Harness Engineering（驾驭工程）**。这三个阶段并不是互相取代的，而是随着问题复杂度和对可靠性要求的提高，**层层递进**的：

1 Prompt Engineering 提示词工程

关注点

微观指令的调优（怎么说模型才能懂）。

局限

单纯的提示词无法解决由于海量结构化数据带来的"注意力稀释"问题。当上下文被塞满时，再好的提示词也无法防止模型破坏代码结构的完整性。

2 Context Engineering 上下文工程

关注点

系统性地管理提供给模型的信息环境（给什么、不给什么、什么时候给）。

进步

通过 Agentic IDE、Plan 模式和 Skills 机制，解决了跨文件、跨会话的记忆和专业知识传递问题，大幅降低了模型的认知负担。

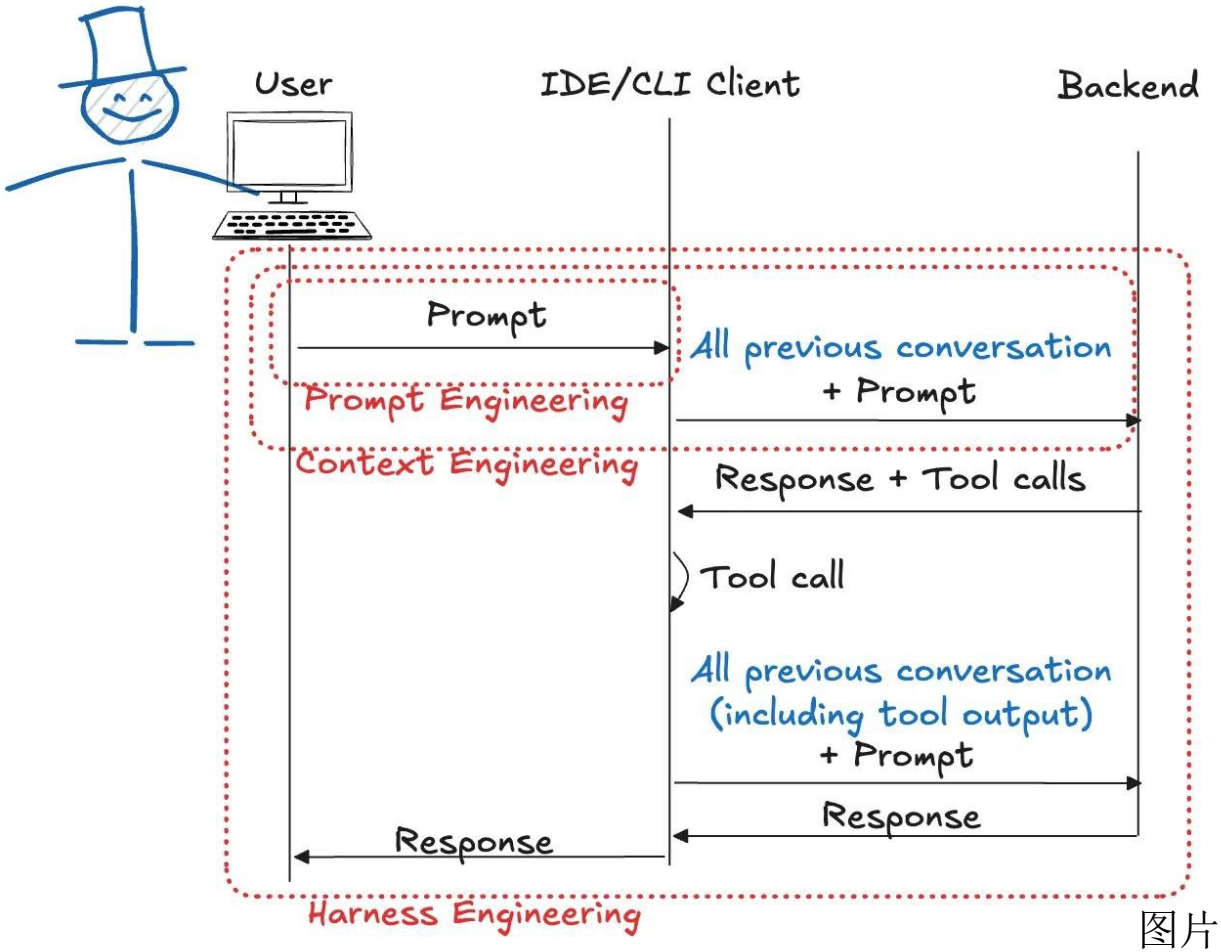
3 Harness Engineering 驾驭工程

关注点

围绕 AI Agent 设计系统架构、执行环境、约束条件和自动化反馈循环，以确保其在生产环境中的绝对可靠性。

> **比喻：** 如果把 LLM 比作一匹充满力量的野马，Harness 就是缰绳和马鞍，让这种原始力量能够被安全、高效地引导和控制。

演进路径



图片引用自：
strategizeyourcareer.com

Harness Engineering

要让 AI Agent 像资深工程师一样稳定地提交代码，必须构建以下"安全护具"：

状态管理
State Management

Agent 不应仅依赖 LLM 会话作为唯一的状态存储。需要建立**确定性的状态机或检查点机制**，当 Agent 执行中断或失败时，可以安全恢复而不会破坏现有系统。

上下文架构
Context Architecture

不要依赖让大模型直接处理复杂的文本，比如 JSON 修改、代码的格式化等。使用**确定性脚本（Deterministic Scripts）**代替大模型直接去处理超大 JSON 或敏感的格式文件，模型只负责输出变更参数，脚本负责执行变更。

护栏机制
Guardrails

在 Agent 的每一步执行前后设置**强约束**（如语法检查、单元测试、静态分析）。如果 Agent 生成的代码破坏了护栏规则，系统应直接拦截，并强制 Agent 重新分析和修改，避免错误扩散到生产环境。

熵管理
Entropy Management

熵（Entropy）在软件工程中指代系统的混乱度或无序程度。AI 生成的代码往往具有增加"系统熵"的天然倾向。需要**自动化的反馈循环**和持续的代码规范审查，确保 Agent 的长期工作不会导致代码库的腐化。

★ **Harness Engineering** 是工程师从**"演示级（Demo）AI"**跨越到**"部署级（Deploy）AI"**的关键分水岭。

Harness Engineering 在 Skill 、 Rule 中的体现

Harness Engineering 的核心理念在日常的生成式开发中，最直接的体现就是我们编写 `Rule` 和 `Skill`的方式：

> **对于确定性的解决方案，尽可能使用脚本（Scripts），而不是依赖大模型去做自由判定。**

为什么在 Skill/Rule 中要倾向于脚本？

① 可靠性与确定性

LLM 是概率模型，处理大文件或复杂校验时容易出错。而脚本（如 Python、Bash）是**确定性的引擎**。

② 职责分离

大模型做决策，脚本做执行：让大模型发挥其强项（理解自然语言意图、提取变更参数），让脚本发挥其强项（执行文件修改、API 交互、环境配置）。

③ 建立护栏机制

将安全检查、敏感操作封装在脚本中，并在 `Skill` 指令里要求大模型**必须**调用该脚本。这就天然形成了一道坚固的护栏。

最佳实践对比

错误的做法（依赖模型判定）

写提示词：在 Rule 中写"修改 `config.json` 时，必须确保没有破坏层级结构，并且务必确保每个节点都要有 `id` 字段"。

Hamess 的做法（依赖脚本执行）

在 Skill 中写"需要更新配置时，提取出用户的变更参数，然后**必须调用 scripts/update_config.py --key X --value Y**，该脚本会自动处理结构并进行 schema 校验"。

生成式软件开发的未来，不仅仅是教大模型写代码，更是构建一套能够**安全驾驭大模型能力**的工程体系。

变化的方向

回顾整个演进过程，可以看出这一演进路线的根本驱动力是：

> **一方面则是我们试图用 AI 解决的工程问题正变得日益复杂，另一方面源于工程界对大模型能力局限性的认知。**

Prompt 阶段

应对单点/代码片段级复杂度

人们最初认为"只要提示词写得足够好，大模型就能写出完美的系统"。但当任务升级为构建完整功能时，现实中遇到了**上下文窗口记忆遗忘**和**逻辑崩塌**的局限。

Context 阶段

应对多文件/项目级复杂度

人们意识到大模型无法凭空记住跨文件的项目背景，因此引入了 RAG、Workspace 感知和 Plan 模式。但面对极高脆弱性的任务，仅仅"给全信息"并不够，大模型依然会**概率性地出错**（如破坏 JSON 结构、忽略边界条件）。

Harness 阶段

应对生产环境/系统级复杂度

当我们要把有一定复杂度的软件部署到生产环境执行高风险任务时，工程师们最终接受了"**大模型本质上是一个概率引擎，它不可能做到 100% 确定性**"这一事实。通过建立护栏、脚本、状态机，将大模型的非确定性与传统代码的确定性剥离开来，**用系统的确定性去兜底**。

总结

CONCLUSION

结语

生成式软件开发正在经历一场深刻的工程化变革。

从最初依赖技巧的**"提示词黑客"**，到强调结构化与闭环管理的**"上下文工程师"**，再到如今致力于构建安全可靠系统的**"驾驭工程师"**。

→ 在课程的下半学期，将根据我们学到的知识，了解 **SDD**、**MCP** 乃至构建具有**"智能"**的 AI Agent。

持续学习　实践探索　未来可期

End

2.氛围编程的关键实践 (7)

一从 Prompt 到 Context到 Harness 的演进